Restaurant Rush

SCMA Group 3

Team Members:

CCDS LEE SHENG KAI
CCDS LEONG XUAN HAO
CCDS POLOYAPOY RON JOSHUA ABES (Apollo)
CCDS VICTORIA HAUGAA

Nanyang Technological University

SC2002 Object Oriented Design and Programming

13th November 2026

1 Overview

Restaurant Rush is a turn-based restaurant management game played on the command line interface (CLI). The player is the manager. Each turn the manager gives one instruction each to the Host, Waiter and Chef, and the Cashier collects payment automatically. With one of each staff member and only two tables, every turn comes down to one question: ***who do we look after first?***

1.1 Base Setting

Rule	**Value**
Length	18 turns, no early ending
Tables	2
Staff	Waiter, Chef and Cashier, plus a Host from Stage 3; one task per turn each
Arrivals	One customer on turns 1, 4, 7, 10, 13 and 16
Menu	Salad $9 (1 unit) Burger $15 (1 unit) Pasta $18 (2 units)
Combos (Stage 3)	Salad + Burger $21.60 (2 units), Salad + Pasta $24.30 (3 units)
To win	After turn 18: - at least 4 paid customers - at least $45 revenue, and - an average satisfaction of at least 60 among paid customers

1.2 What happens in one turn

Figure 1.1 shows the order of the steps in one turn.

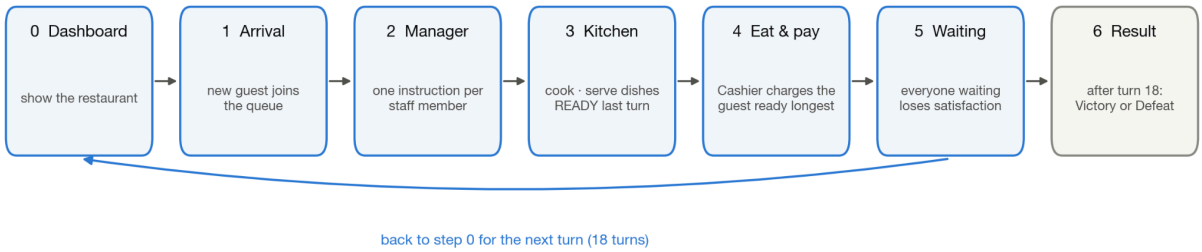


Figure 1.1: One turn of Restaurant Rush.
Steps 0–5 repeat every turn for 18 turns. The result check (step 6) runs once, after turn 18.

Step	What happens	Handled by
0	The dashboard shows the restaurant	Dashboard
1	A new customer arrives and joins the queue	Game.beginTurn → ArrivalSchedule, Restaurant.admit
2	The manager gives each staff member one instruction	ConsoleUI → DirectedStaff.perform
3	The kitchen cooks. A dish that becomes READY this turn can only be served next turn.	Order.prepareUnit, Order.serve
4	Customers finish eating, and the Cashier takes one payment (oldest first)	Customer.finishEating, Cashier.collectPayment
5	Everyone still waiting loses satisfaction. A customer who reaches 0 leaves.	Customer.loseSatisfaction, Restaurant.depart
6	After turn 18: Victory or Defeat	Objective.evaluate → GameResult

1.3 The three stages

Tag	What it adds	Customers	Rule checks
stage-1	Core game: Regular customers, Menu, Waiter, Chef, Cashier, Both endings	All six are Regular	12
stage-2	VIP and Critic customers	Ben becomes a Critic, Chloe becomes a VIP	16
stage-3	Host, Combo Meals, Happy Hour	Same as Stage 2	22

To run it: ./run.sh to play, ./run.sh --script scripts/stage3-victory.txt to replay the victory run and ./test.sh for the rule checks. It needs Java 17 or newer and no external libraries.

1.4 The six customers

The same six customers arrive in every game.

#	Customer	Arrives on turn	Type (Stages 2–3)	Satisfaction lost per waiting turn	Stage 3 order
1	Ada	1	Regular	8	Salad
2	Ben	4	Critic	12	Salad + Pasta Combo
3	Chloe	7	VIP	5	Pasta
4	Dev	10	Regular	8	Salad + Burger Combo
5	Elif	13	Regular	8	Salad + Pasta Combo
6	Farid	16	Regular	8	Salad

In Stage 1, all six are Regular and lose 8 per waiting turn.

Farid arrives too late to pay within 18 turns, so at most five customers can pay.

1.5 The sequence diagram scenario: Chloe and Ben

The sequence diagrams (Section 3) replay the Stage 3 victory run. They follow **Chloe (#3, VIP)** from arrival on turn 7 to payment on turn 11, then the result check after turn 18.

Ben (#2, Critic) is the only other guest in the restaurant on turns 7–9. Ben has no lifeline, which keeps the diagrams focused on Chloe, but appears in the notes to explain four steps:

1. Happy Hour recovery
2. The satisfaction loss at the end of a turn
3. Which customer the Waiter serves
4. Which customer the Cashier charges first

Details	Chloe (#3)	Ben (#2)
Type	VIP	Critic
Satisfaction lost per waiting turn	5	12
Special behaviour	10% off the bill	Orders the most elaborate dish and leaves a star review
Arrives	Turn 7	Turn 4
Table	1	2
Order	Pasta (2 units)	Salad + Pasta Combo (3 units)
Price locked	$12.96 ($18, less 10% VIP, less 20% off Happy Hour)	$24.30 (combo = 90% of $27)
Served	Turn 9	Turn 8
Pays	Turn 11	Turn 10
Final satisfaction	95	82
In the diagrams	Main lifeline, all six parts	Notes only

Figure 1.2 shows how their satisfaction changes from turn 4 to 11, and Figure 1.3 shows the same turn 7 as it appears in the game.

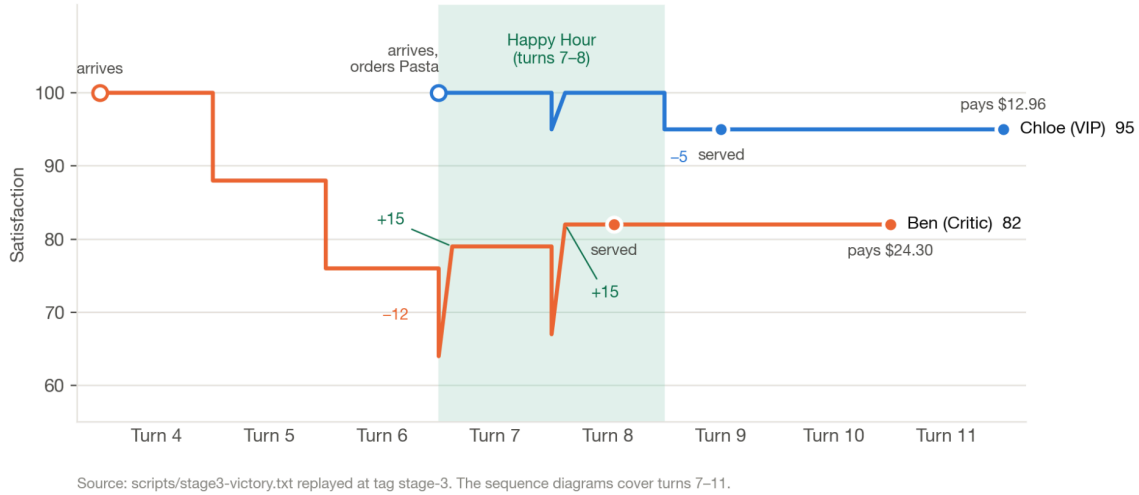


Figure 1.2: Chloe's and Ben's satisfaction, turns 4–11 (Stage 3 victory run).
Ben (Critic) loses 12 for each turn he waits and Chloe (VIP) loses 5. Happy Hour (green, turns 7–8) gives every waiting guest +15. Both stop losing satisfaction once they are served.

Turn	Diagram part	Chloe	Ben
7	1-2	- Arrives - Is seated at table 1 - Waiter takes the Pasta order at $12.96 - Turn end: 100 → 95	- Happy Hour +15 (64 → 79) - His combo waits because the Waiter took Chloe's order first. - Turn end: 79 → 67
8	3	- Happy Hour: back to100. - Pasta becomes READY but cannot be served until next turn. - Turn end: 100 → 95	- Happy Hour: 67 → 82 - Waiter serves his combo (not drawn)
9	4	- Pasta served, so satisfaction stops falling	- Finishes eating, ready to pay
10	5	- Finishes eating, ready to pay from this turn	- Pays $24.30, having been ready longer
11	5	- Pays $12.96 - Table 1 is freed	- Already gone
After 18	6	Victory: 5 paid, $92.16 revenue, 82.6 average satisfaction	

Figures 1.4 and 1.5 show the Happy Hour recovery on turn 8 and the payment order on turns 10 and 11.



Figure 1.3: Turn 7 in the game (Sequence Diagrams 1 and 2).

Chloe arrives and the manager switches Happy Hour on. The Host seats her and the Waiter takes her Pasta order at $12.96, even though Ben's combo is already READY. At the end of the turn the same satisfaction loss costs Ben 12 and Chloe 5.

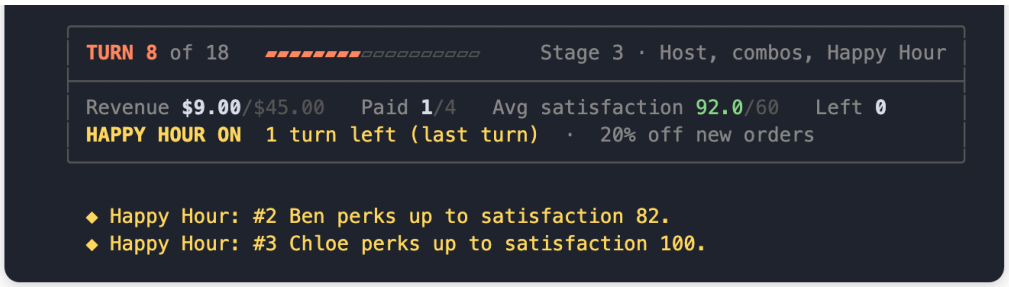


Figure 1.4: Start of turn 8 (Sequence Diagram 3).

Before any staff member acts, Happy Hour lifts Ben to 82 and Chloe back to 100.

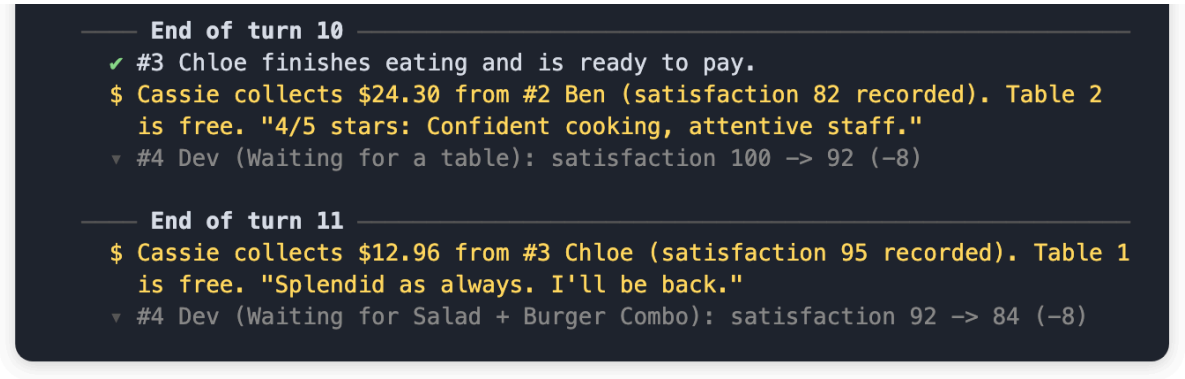
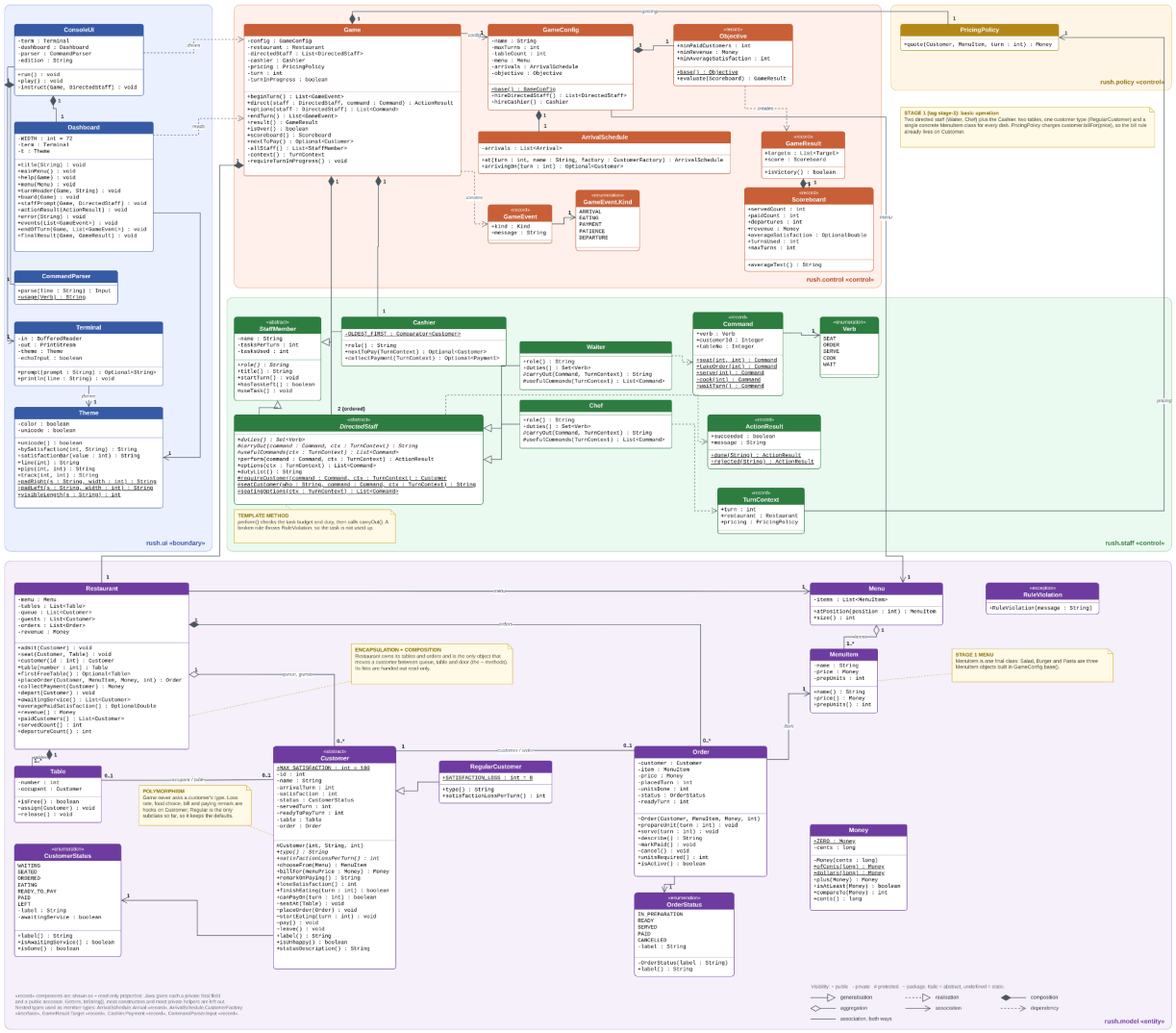


Figure 1.5: End of turns 10 and 11 (Sequence Diagram 5).
Ben has been ready to pay longer, so the Cashier charges him first ($24.30). Chloe pays $12.96 on turn 11.

The one manager decision that changes the outcome in this scenario is switching Happy Hour on at turn 7. Section 5.3 shows what it costs and what it gains.

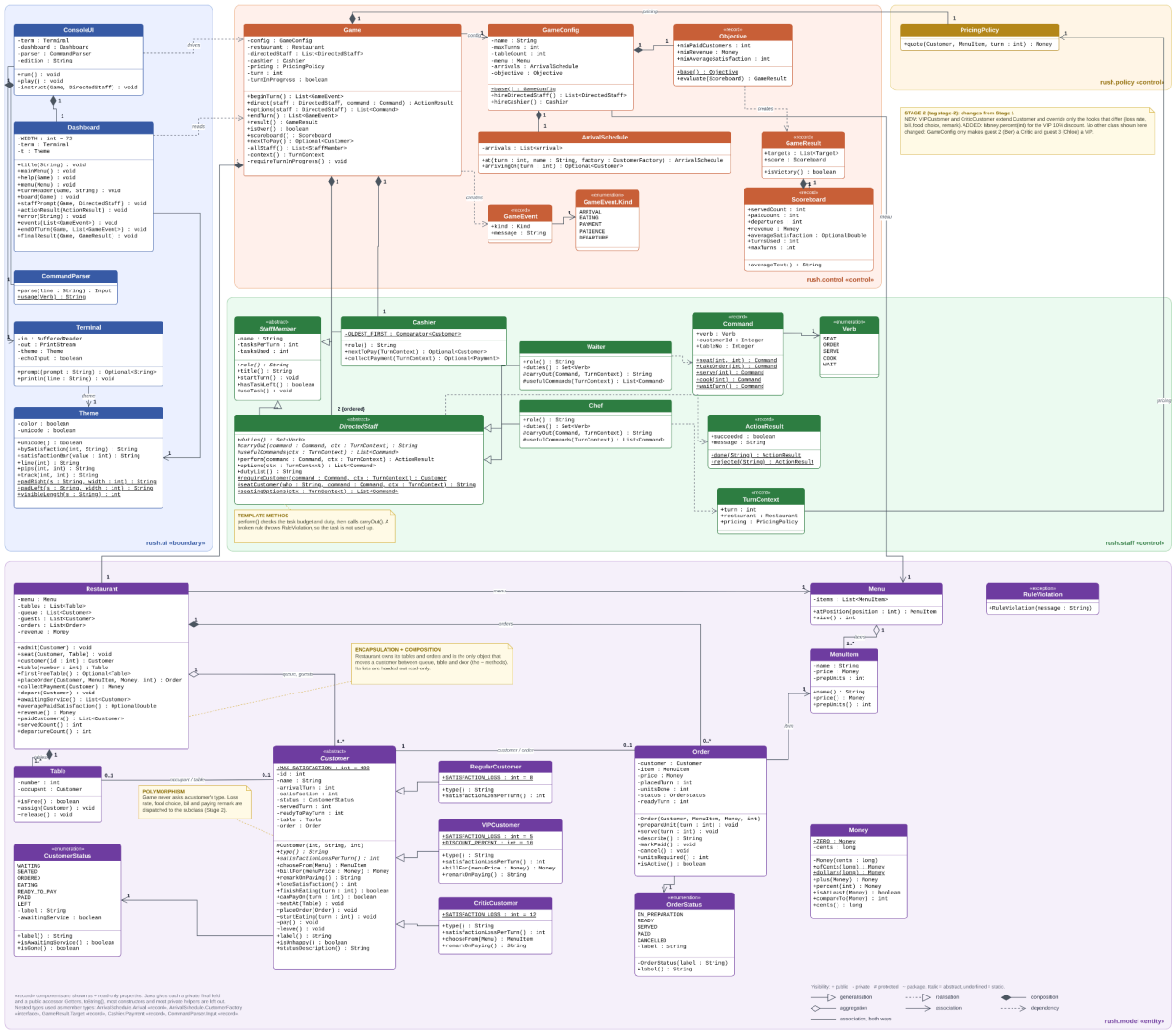
Class Diagram

Stage 1



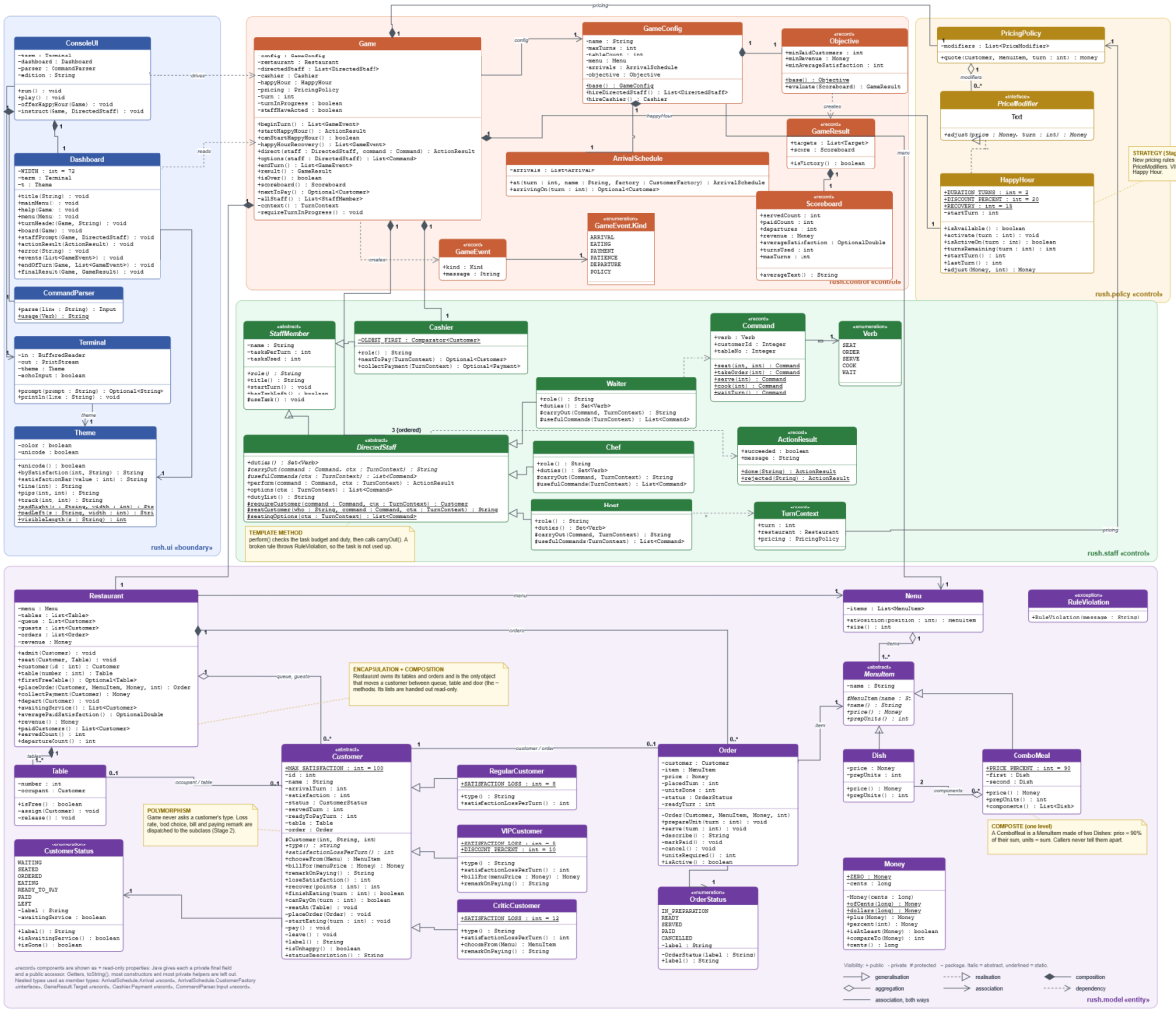
Concept	Example	Why it is meaningful
Encapsulation	Restaurant keeps private variables tables, queue, guests, orders and revenue Customer.seatAt(), placeOrder(), pay() and leave()	No other class can access the private variables and seat,bill or remove a customer without Restaurant.
Encapsulation	Customer keeps satisfaction private, it can only change through loseSatisfaction()	No other class can access and change the private satisfaction.
Polymorphism	Game.direct(staff, command) calls staff.perform() on a DirectedStaff. perform() is final and calls the abstract carryOut(), duties() and usefulCommands(), which Waiter and Chef override	Game holds a list DirectedStaff and treats the Waiter and Chef the same. The task budget and duty checks are written once in perform().
Polymorphism	staffMember.role() is abstract and overridden by Waiter, chef and Cashier. The title() builds the display name from role()	Each role names itself hence no caller needs to know the staff classes.
Polymorphism	Game.direct(staff,command) calls staff.perform() on a DirectedStaff. perform() calls the abstract carryOut(), duties and usefulCommands(), which Waiter and Chef override	Game holds a list DirectedStaff and treats the Waiter and Chef the same. perform() calls the abstract carryOut(), duties and usefulCommands(), which Waiter and Chef override
Composition	Game is composed of its Restuarant , PricingPolicy, DirectedStaff and its Cashier.	These parts can only exist within this game.
Composition	Restaurant is composed of its Tables and its Orders	Tables and Orders cannot exist outside of the restaurant.

Stage 2


ConsoleUI
Dashboard
CommandParser
Terminal
Theme
rush.ui «boundary»
Game
GameConfig
Objective
PricingPolicy
ArrivalSchedule
GameResult
GameEvent
GameEventKind
Scoreboard
rush.control «control»
StaffMember
Cashier
Waiter
Chef
DirectedStaff
Command
Verb
ActionResult
TurnContext
rush.staff «control»
Restaurant
Table
Customer
RegularCustomer
VIPCustomer
CriticCustomer
CustomerStatus
Menu
MenuItem
RuleViolation
Order
OrderStatus
Money
rush.model «entity»

Concept	**Example**	**Why it is meaningful**
Encapsulation	The VIP discount is kept inside VIPCustomer.billFor(), that returns menuPrice.percent(100-DISCOUNT_PERCENT) and PricingPolicy still only calls customer.billFor()	The discount rule belongs specifically to the VIP and no other class knows/have it.
Encapsulation	The Critic always orders the menu item that takes the most preparation unit, This rule is stored private as MOST_ELABORATE in CriticCustomer and only its chooseFrom() uses it	No other class can access MOST_ELABORATE variable
Polymorphism	RegularCustomer,VIPCustomer and CriticCustomer override the Customer. Regular losses 8 satisfaction per turn, VIP loss 5 at the expense of 10% billFor() discount, Critic losses 12 and picks the most elaborate dish chooseFrom() with star review in remarkOnPaying()	Game,Waiter and PricingPolicy call loseSatisfaction(),chooseFrom(),billFor() and remarkOnPaying() on an customer.
Composition	Unchanged from stage 1	Since the new customer types fit into the existing codes, so no new links were needed

Stage 3


ConsoleUI
Dashboard
CommandParser
Terminal
Theme
rush.ui «boundary»
Game
GameConfig
ArrivalSchedule
GameEvent
GameEvent.Kind
rush.control «controls»
PricingPolicy
PriceModifier
HappyHour
rush.policy «controls»
StaffMember
Cashier
Waiter
Chef
Host
DirectedStaff
Command
Verb
ActionResult
TurnContext
rush.staff «controls»
Restaurant
Menu
MenuItem
Dish
ComboMeal
Table
Customer
RegularCustomer
VIPCustomer
CriticCustomer
Order
OrderStatus
CustomerStatus
Money
RuleViolation
rush.model «entity»

Concept	Example	Why it is meaningful
Encapsulation	HappyHour keeps startTurn private. Its timing rules are behind isAvailable(), activate(), isActiveOn() and turnsRemaining().	The activate only once per game, two turn lives inside HappyHour.
Encapsulation	ComboMeal keeps first and second private. components() returns an unmodifiable List.of(first,second) and the 90% price is calculated inside price()	A combo cannot be amended after it is built.
Polymorphism	MenuItem became abstract with abstract price() and prepUnits(), overridden by Dish and ComboMeal.	A combo is cooked, priced and served exactly like a dish.
Polymorphism	Host is a DirectedStaff subclass with its own duties(),carryOut(), and reusing the shared seatCustomer	The Host is added only in GameConfig, and the Game treats the Host like the other staffs
Composition	ComboMeal aggregates two Dishes, a combo is built from two dishes. It is drawn as aggregation since the same Salad Dish is on the menu and in both combos	The combo is a whole consisting of Salad that is also shared with the menu. Hence they do not “live” only in combo.

UML Sequence Diagrams

Scenario Background

The scenario we have chosen for our UML sequence diagram follows VIP customer number 3, Chloe, from her arrival on turn 7 to her payment on turn 11, and then the result check after turn 18. The UML sequence diagrams have been split into 6 parts for readability purposes.

Note: Ben is guest #2, a critic who arrives on Turn 4. He is still at table 2 waiting for his salad + pasta combo when Chloe arrives, so he is the only other guest in the restaurant at that point. He has no lifeline in the diagram as we have decided to focus on the FULL scenario involving Chloe. However, there are a few labels and notes that mention him which are necessary to explain certain steps: Happy Hour Recovery, the loss of satisfaction at turn end, the Waiter's choice of whom to serve, and the Cashier's payment order. To see more details of Ben, refer to the table in the overview.

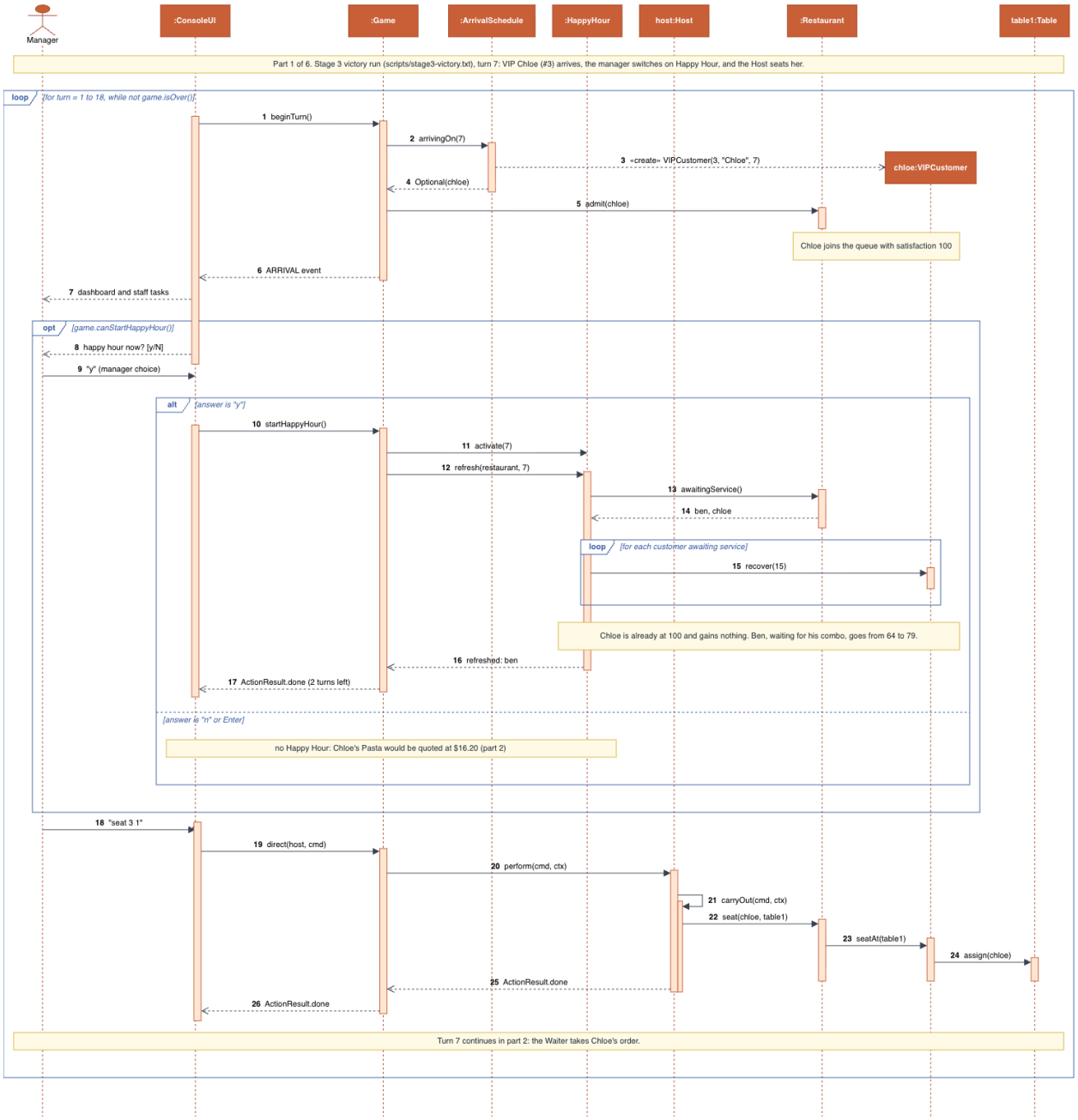
Notation

In each UML diagram, solid arrows are calls while dashed arrows are returns. A dashed <<create>> arrow ends on the classifier of the new object, which is drawn at that point instead of at the top with the other classifiers. The narrow vertical boxes on the lifelines are the activation boxes or execution specifications which show the execution of a unit of behaviour or action within the lifeline, i.e. when an object is running a method. An arrow that loops back onto the same lifeline is an object calling one of its own methods, i.e. a self message/recursive message, and it gets its own nested activation box so that we can see where the message is coming from. Every guard in the diagram is a real condition that is implemented in the code.

Fragment	**Guard**	**Sequence Diagram Number**	**Meaning**
loop	For turn 1 to 18, while not game.isOver( )	all	One pass per turn
opt	game.canStartHappyHour( )	1	The prompt only appears if Happy Hour is not on and no staff member has acted

alt	If answer is “y”, else if answer is “n” or Enter	1	The manager’s choice can either be to switch Happy Hour on or keep the $16.20 price
loop	For each customer awaiting service	1,2,3	Happy Hour recovery and turn end satisfaction loss
loop	For each PriceModifier	1	Apply VIP discount (10%) then followed by Happy Hour Discount (20%), in order
loop	While waiter.hastaskLeft( )	4	Manager chooses again after a rejected serve
alt	If status is READY and readyTurn < turn, else if still cooking	4	The serve succeeds, or it is rejected
opt	If oldest customer can pay and the cashier’s task is unused	5	A payment only happens when the customer is eligible and when the cashier has not used its task in the turn
alt	If result.isVictory( ), else	6	Displays the victory or defeat message

Sequence Diagram 1

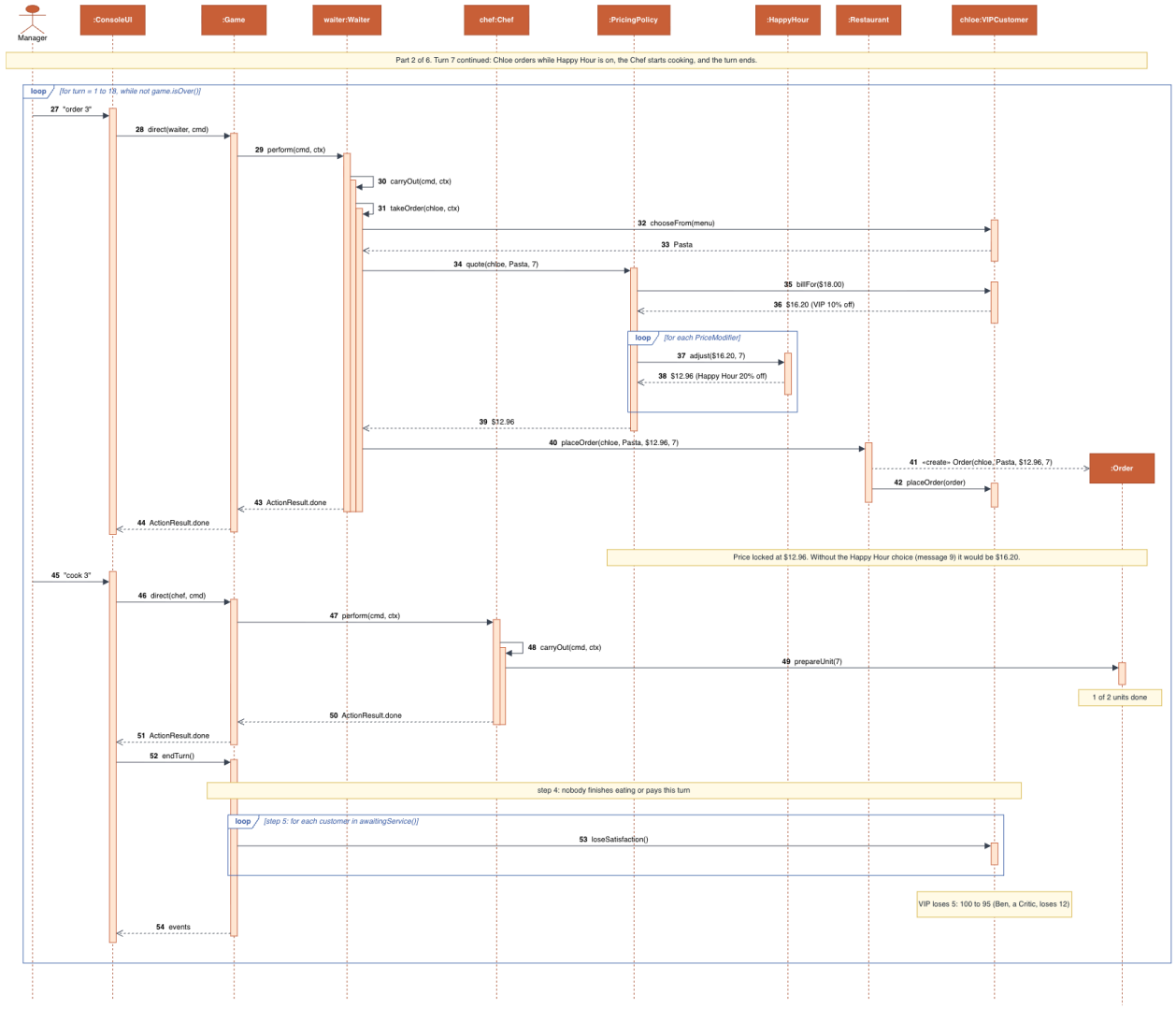


Sequence Diagram 1 WalkThrough

Requirement	Turn	Messages	What Happens
Customer Arrival	7	1-7	Game.beginTurn calls ArrivalSchedule.arrivingOn(7)

Sequence Diagram 1 WalkThrough			
			which creates VIPCustomer Chloe. Fame.beginTurn then calls Restaurant.admit to put Chloe in the queue at satisfaction 100
Stage 3 Behaviour: Happy Hour	7	8-17	In our scenario, the manager answers y. Game.startHappyHour calls HappyHour.activate(7), then refresh which calls Restaurant.awaitingService( ) to see who is awaiting service and gets Ben and Chloe. It then sends recover(15) to each waiting customer but since Chloe has just arrived at 100, she gains nothing and so only Ben comes back as refreshed and his satisfaction increases from 64 to 79
Host assignment	7	18-26	“seat 3 1” means sit customer 3 at table 1. Game.direct passes it to the Host where Host.perform calls its own carryOut method which calls Restaurant.seat to take Chloe out of the queue. Customer.seatAt and Table.assign records her seat on both Chloe and table 1.

Sequence Diagram 2

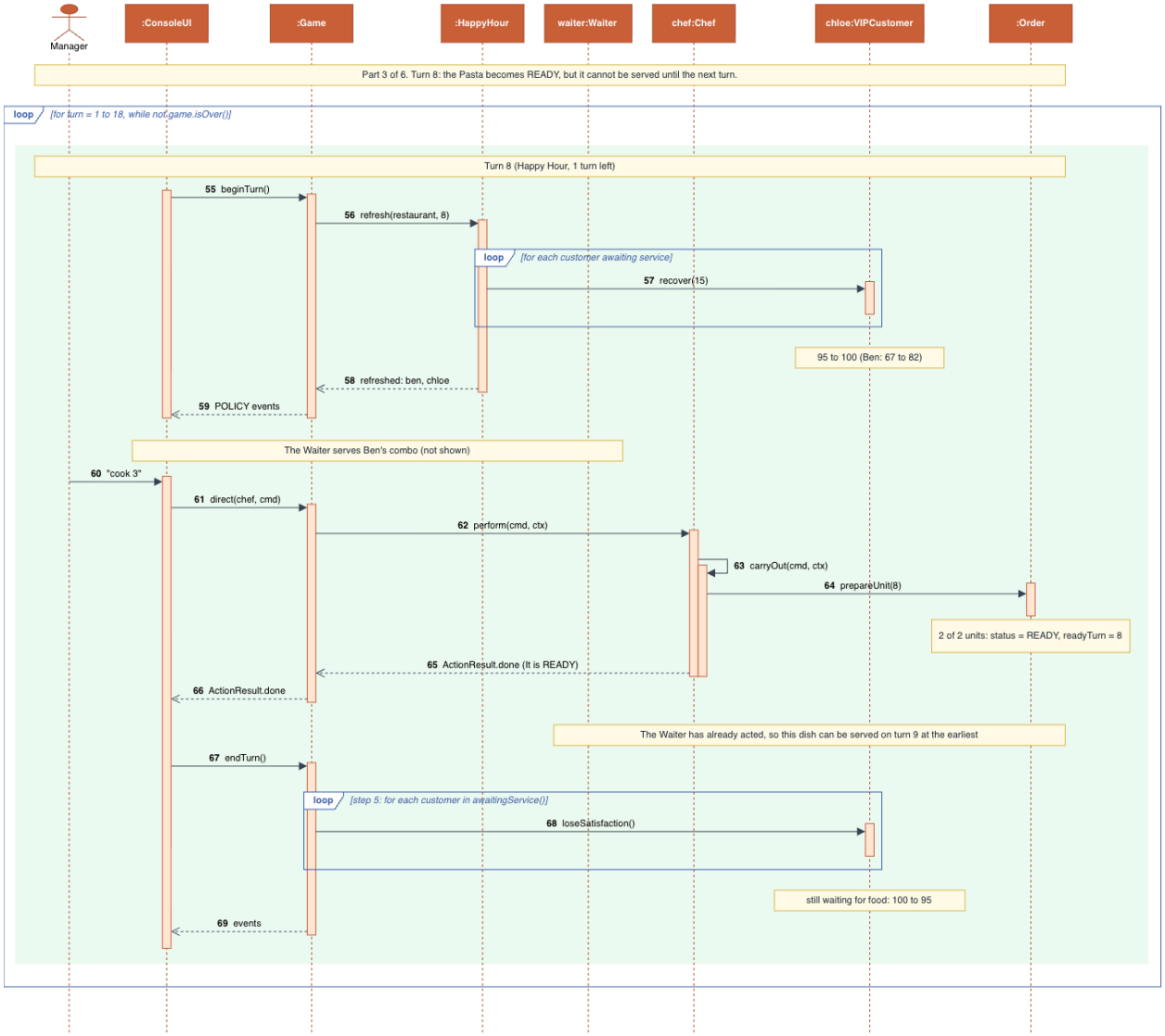


Sequence Diagram 2 WalkThrough

Requirement	Turn	Messages	What Happens
Customer Order	7	27-44	The manager sends the waiter to take Chloe's order rather than serve Ben's combo, which is already waiting. The waiter calls chloe.chooseFrom(menu) and it receives Pasta. PricingPolicy.quote calls billFor then HappyHour.adjust. This ensures the discounts are applied in order.

Sequence Diagram 2 WalkThrough			
			Restaurant.placeOrder creates the order.
Chef Assignment	7	45-51	The Chef calls Order.prepareUnit(7). 1 of the 2 units of pasta is done on the same turn the order is placed
Waiting and abandonment	7	52-54	Game.endTurn sends the same loseSatisfaction( ) to every customer awaiting service. Chloe, a VIP, loses 5 points and her satisfaction decreases from 100 to 95. While Ben, a critic, receives the same message and loses 12 points, his satisfaction decreases from 79 to 67.

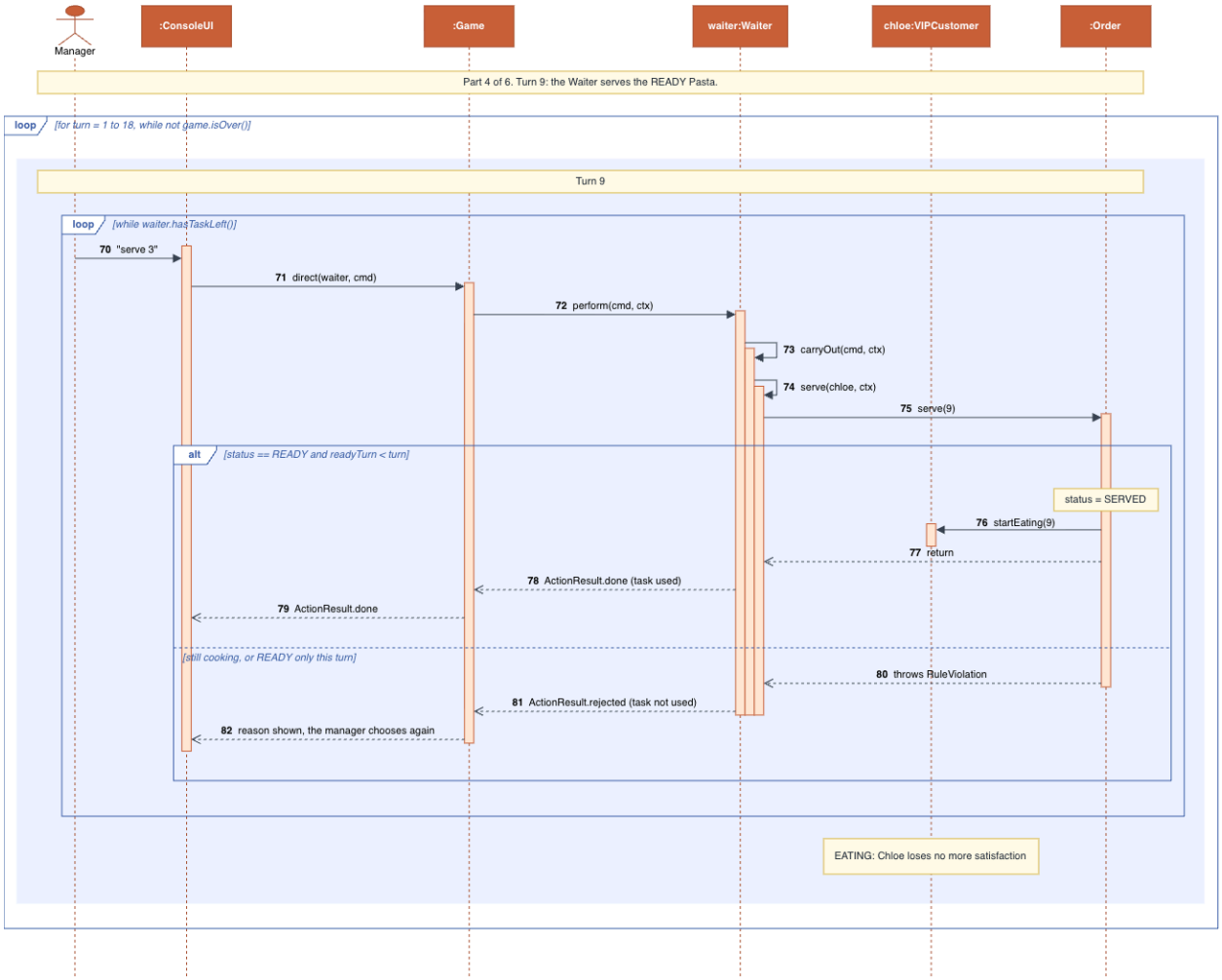
Sequence Diagram 3



Sequence Diagram 3 WalkThrough			
Project Requires	**Turn**	**Messages**	**What Happens**
Happy Hour Recover	8	55-59	Game.beginTurn calls HappyHour.refresh(restaurant, 8) before anyone acts. Refresh calls recover(15) to each customer still waiting for service and Chloe's satisfaction increases from 95 to 100 (satisfaction capped at 100). It returns who recovered, and Game

Sequence Diagram 3 WalkThrough			
			passes them to the UI as POLICY events.
READY	8	60-66	The waiter serves Ben's combo first (this is not shown in the diagram), which uses up the waiter's one task. "cook 3" is the input by the manager. Game.direct calls Chef.perform, which calls the Chef's own method carrOut and that calls Order.preapareUnit(8) to cook one more unit. That was the pasta order's second and last unit, so it becomes READY with readyTurn = 8, and a done result is returned to the UI. The pasta will only be served in the next turn (turn 9) despite it being ready in turn 8
Waiting and Abandonment	8	67-69	Game.endTurn sends loseSatisfaction( ) to each customer still waiting for service. Ben has been served, so Chole is the only one left and her satisfaction from 100 to 95. The events return to the UI.

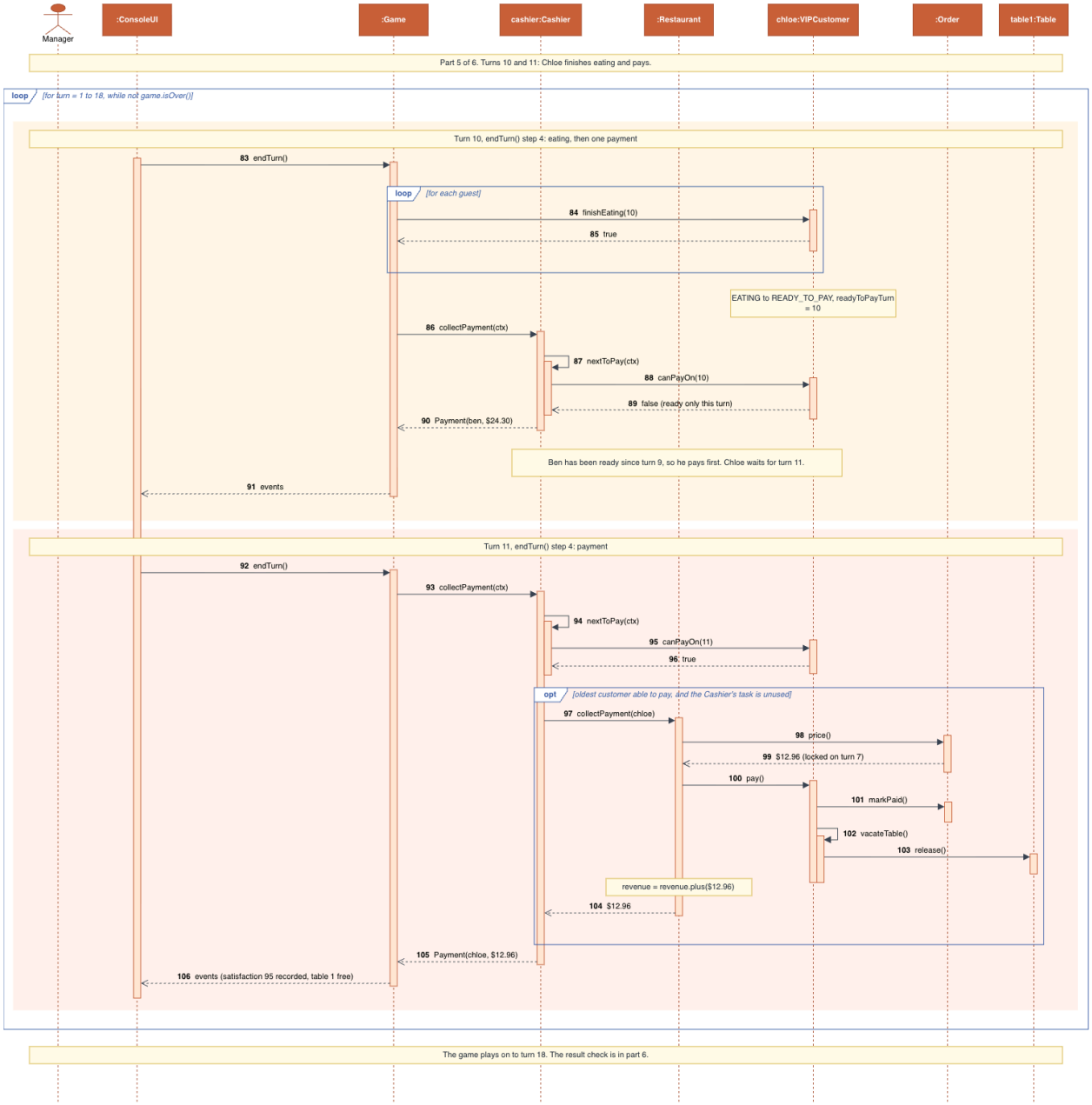
Sequence Diagram 4



Sequence Diagram 4 WalkThrough			
Project Requires	**Turn**	**Messages**	**What Happens**
SERVED (the call)	9	70-75	The manager inputs “serve 3” . Game.direct calls Waiter.perform, which calls the waiter’s own method carryOut, and that calls its own method serve. Serve then calls Order.serve(9), which accepts the dish only if its status is READY and readyTurn < turn

Sequence Diagram 4 WalkThrough			
SERVED (accepted)	9	76-79	Chloe's pasta has been READY since turn 8, so it is accepted. The order becomes SERVED and calls chloe.startEating(9), which sets Chloe to EATING, so she loses no more satisfaction. The waiter's task is used and a "done" result is returned up to the UI.
SERVED (rejected, the alternative)	9	80-82	If the dish were still cooking, or only READY this turn and not the turn before, Order.serve would throw RuleViolation instead. The waiter's task is not used, the UI shows the reason and lets the manager choose again

Sequence Diagram 5

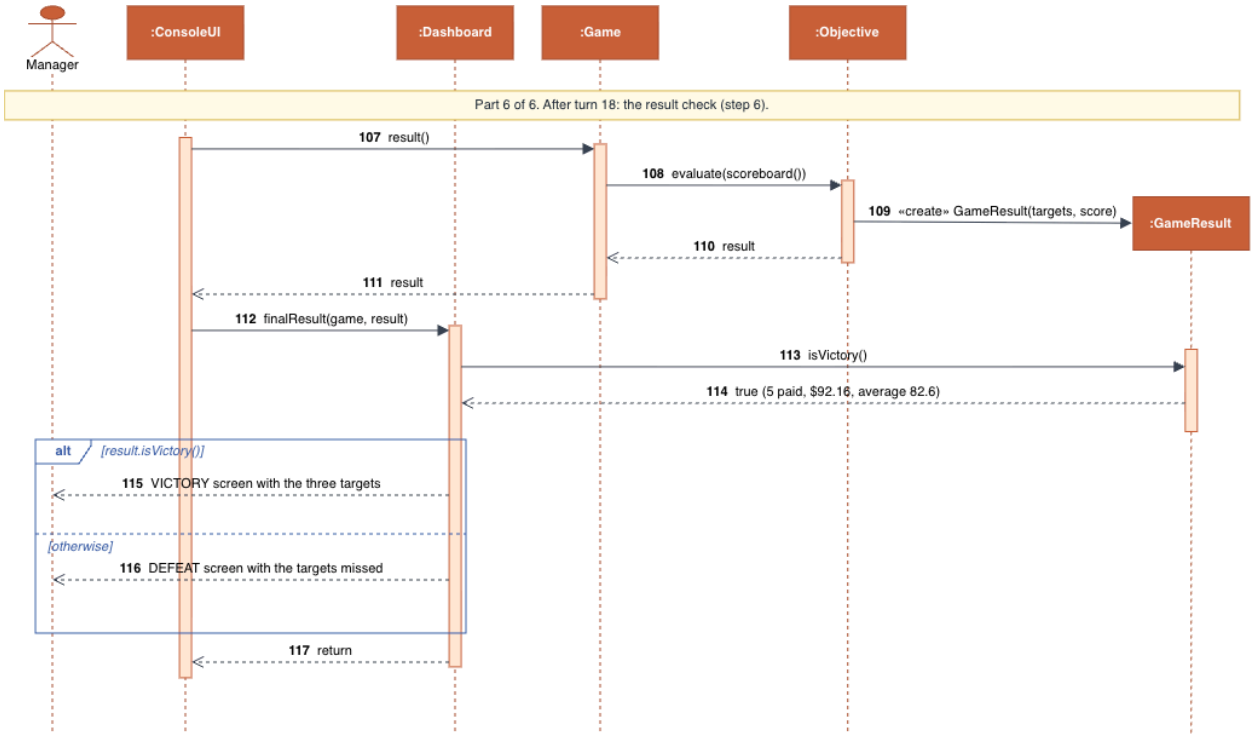


Sequence Diagram 5 WalkThrough			
Project Requires	**Turn**	**Messages**	**What Happens**
Eating	10	83-85	Game.endTurn calls chloe.finishEating(10). Chloe was

Sequence Diagram 5 WalkThrough			
			served on turn 9 and has now eaten for one full turn, so she moves to READY_TO_PAY with readyToPayTurn = 10 and the call returns true.
Payment (turn 10)	10	86-91	Game calls Cashier.collectPayment, which calls its own method nextToPay. nextToPay calls canPayOn(10) on Chloe. Chloe became ready only this turn, so she answers false, because a customer pays the turn after becoming ready. Ben has been ready since turn 9 so the cashier takes charges Ben and the events return to the UI
Payment (turn 11, who pays)	11	92-96	Game calls Cashier.collectPayment, which calls its own method nextToPay. nextToPay calls canPayOn(11) on Chloe. Now canPayOn(11) is true for Chloe, because she has been ready since turn 10. She is the oldest customer able to pay, so the Cashier charges her.
Payment (turn 11, charging Chloe)	11	97-103	The cashier calls Restaurant.collectPayment(chloe). Restaurant calls the order's price( ) and gets the $12.96 locked on turn 7, then calls chloe.pay( ). pay( ) calls markPaid( ) on the order and then its own method vacateTable( ), which calls release( ) on table1.
Payment (turn 11, revenue and result)	11	104-106	Restaurant adds $12.96 to revenue and returns it to the cashier. The cashier returns Payment(chloe, $12.96) to Game, and the events (satisfaction 95 recorded, table 1 free) return to the UI

Sequence Diagram 6

Note: Sequence Diagram 6 happens after turn 18 is finished and the turn loop has exited (game.isOver( ) is true), so it sits outside the loop frame used in parts 1-5. This covers the whole game rather than the scenario involving Chloe alone.



Sequence Diagram 6 WalkThrough			
Project Requires	**Turn**	**Messages**	**What Happens**
Result check	After 18	107-117	Game.result calls Objective.evaluate(scoreboard), which creates a GameResult object. isVictory( ) picks the Victory or Defeat screen.

Design Discussion

4.1 Key abstractions and their responsibilities

Abstraction	Layer	Responsibility
Game	control	Runs the six steps of a turn in order. The rules it triggers live in domain and staff objects.
GameConfig, ArrivalSchedule	control	The base setting: turns, tables, menu, who arrives when and which staff are hired. Each stage registers its additions here.
Objective, Scoreboard, GameResult	control	The targets, end-of-turn figures and the Victory or Defeat verdict.
StaffMember, DirectedStaff	staff	The per-turn task budget, and the template method that checks and carries out an instruction.
Waiter, Chef, Host	staff	What one role may do, and which commands are useful right now (shown as hints).
Cashier	staff	Takes one payment per turn from the customer who has been ready the longest.
PricingPolicy, PriceModifier, HappyHour	policy	Locks the bill when an order is taken. Happy Hour's once-per-game life cycle, discount and recovery.
Restaurant	model	Keeps the queue, tables, guests, orders and revenue consistent with each other.
Customer, RegularCustomer, VIPCustomer, CriticCustomer	model	The visit life cycle and satisfaction (base class); patience, food choice and bill (subclasses).
Order	model	The locked price, cooking progress, the READY then SERVED rule, and cancellation.
MenuItem, Dish, ComboMeal, Menu	model	What can be ordered. A combo works out its price and units from its dishes.
Money, RuleViolation	model	Exact cent arithmetic, and a readable reason when a move breaks a rule.
ConsoleUI, Dashboard, CommandParser, Terminal, Theme	boundary	Prompts, drawing the state, checking the shape of the command, input/output, and colours with an ASCII fallback.

4.2 Design principles in the code

Single Responsibility: Each rule lives in one place. “A dish finished this turn is served next turn at the earliest” is only checked in `Order.serve`. The Waiter just calls it, and the UI just prints the `RuleViolation` message. `CommandParser` only checks that `seat 2 1` is well formed, never whether the move is legal, and `Dashboard` only reads the game.

Open/Closed: Going from Stage 1 to Stage 2 added the VIP and Critic without touching `Game`, `Customer`, `Restaurant`, `Order` or any staff class (`git diff stage-1 stage-2 -- src/rush/contro/Game.java src/rush/staff` is empty). In Stage 3 the Host is a new DirectedStaff added into GameConfig, when the console loops over `game.directedStaff()` picks up without needing to be changed (ConsoleUI changed only for the Happy Hour prompt and auto-wait, see Section 6). A combo is a new kind of MenuItem so no changes to *Order*, *Chef* or *Waiter*.

Liskov Substitution: The status transitions (`seatAt`, `placeOrder`, `startEating`, `pay`, `leave`) are written once in `Customer` and no subclass overrides them. The overrides keep the base contract: a loss rate of at least 0, an item from the given menu, and a bill no higher than the menu price. A rule check runs a VIP through exactly the same flow as a Regular. A `ComboMeal` can stand in for any `MenuItem`, and the Critic’s choice and the Chef’s unit counting work unchanged.

Interface Segregation: The Cashier takes no instructions, so it is a `StaffMember` but not a `DirectedStaff`, and it does not have to implement `duties()` or `carryOut()`. `ConsoleUI` only uses the small `DirectedStaff` API, and `PriceModifier` has a single method.

Dependency Inversion (partly): `PricingPolicy` depends on `PriceModifier` rather than `HappyHour`, and `ArrivalSchedule` depends on a `CustomerFactory` interface, so it never names a concrete customer class. However, `Game` still creates `HappyHour` and `PricingPolicy` itself. Passing them in through `GameConfig` would be the next step if more policies were added.

4.3 Patterns used

Pattern	Where	Why
Template Method	DirectedStaff.perform	The task-budget check, the duty check and "a rejected move costs nothing" are written once for every role.
Composite (one level)	MenuItem, Dish, ComboMeal	A combo is a menu item built from two Dish leaves, so its price and units are derived rather than copied. A combo cannot contain another combo.
Strategy	PricingPolicy with PriceModifier	Restaurant-wide pricing rules plug in after the customer's own discount, in a fixed order.
Command (simple)	Command Record	Separates what the manager typed from who checks it, so a script is just a file of typed commands.
Factory References	ArrivalSchedule.at(4, "Ben", CriticCustomer::new)	Each stage registers its customers without the schedule knowing the customer classes.

4.4 Trade-offs

Decision	Benefit	Cost
Domain objects throw *RuleViolation* with a message for the player	One source of truth per rule, and the message shown is the rule itself. The UI never repeats a "can I?" check	Exceptions on an expected path. Only *DirectedStaff.perform* and *Game.startHappyHour* catch them.
Customer abstract from Stage 1, MenuItem concrete until Stage 3	Customer types were certain to come. Combos were the only reason for a second kind of menu item, so we waited until we needed it	Stage 3 had to change MenuItem and its setup in GameConfig. The callers did not change.
Events are short English Strings	Simple, and the UI styles them by GameEvent.Kind without parsing	Tests have to compare text. A structured event per kind would be cleaner.
The dashboard colours customer types by name	Presentation stays out of the domain	There is a small switch on *type( )* in the UI, used only for colour. The rules have no type switch.
Staff with nothing useful to do wait automatically (Stage 3)	Fewer pointless prompts once there are three roles per turn	A UI change between stages (documented in Section 6).

4.5 An alternative we considered: the State pattern for customers

A Customer moves through *WAITING* → *SEATED* → *ORDERED* → *EATING* → *READY_TO_PAY* → *PAID*, or *LEFT*. We sketched one class per state, each implementing `seat()`, `order()`, `serve()` and `endOfTurn()` and rejecting moves it does not allow. That puts every transition in one place per state. We kept the enum because the life cycle is strictly linear, and the type behaviours (loss rate, bill, food choice) cut across every state. We would have had seven state classes and three customer types to keep in sync. An enum with guarded methods in one abstract class is shorter, easier to test, and keeps the polymorphism in the customer types.

We also rejected a `CustomerType` enum with switches in the controller, because the brief asks for dispatch through the customer objects. We also rejected a Decorator chain for prices, because "VIP first, then Happy Hour" reads more clearly as an ordered list in `PricingPolicy`.

5 How the player's choices change the result

The base setting has no randomness**, so** any difference in the **result** comes from the manager. We replayed the real `Game` with fixed strategies built from four decisions.

Decision	**Options**	**What it affects**
Waiter priority	Which of *seat*, *order* or *serve* **comes** first when **several are** possible	Whether customers wait in the queue, at a table, or for food
Chef timing	Cook an order on the same turn it is taken or one turn later	How soon a dish becomes READY
Host (Stage 3)	Let the Host seat customers, or leave seating to the Waiter	Frees the Waiter's single task for ordering and serving
Happy Hour (Stage 3)	Which turn to switch it on, or never	Lower prices against higher satisfaction

5.1 Stages 1 and 2: Waiter priority decides the game

Each **result** shows paid customers, revenue and average satisfaction.

Waiter priority	**Chef cooks same turn**	**Stage 1**	**Stage 2**
Serve → order → seat	Yes	Victory: Paid customers: 5 Revenue: $66.00 Average satisfaction: 79.2	Victory: Paid customers: 5 Revenue: $67.20 Average satisfaction: 76.0
Serve → order → seat	No	Victory: Paid customers: 4 Revenue: $51.00 Average satisfaction: 66.0	Victory: Paid customers: 4 Revenue: $52.20 Average satisfaction: .0
Seat → order → serve	No	Victory: Paid customers: 4 Revenue: $51.00 Average satisfaction: 60.0	Victory: Paid customers: 4 Revenue: $52.20 Average satisfaction: 62.5
Seat → serve → order	No	Defeat: Paid customers: 3 Revenue: $42.00 Average satisfaction: 60.0	Defeat on all three targets: Paid customers: 3 Revenue: $43.20 Average satisfaction: 59.3

The Stage 2 defeat misses all three targets, partly because Ben, now a Critic, finishes at 40 instead of 60.

5.2 Stage 3: the Host and Happy Hour change the best strategy

Figure 5.1 compares the six strategies against the three targets. Figures 5.2 and 5.3 show the real end screens for the best and worst of them.



Figure 5.1: The six Stage 3 strategies against the three targets.
Red bars miss their target. Only Rows E and F lose, and both play without the Host.

Row	Waiter priority	Chef same turn	Host used	Happy Hour	Paid	Revenue	Avg	Result
A *(our script)*	order → serve → seat	Yes	Yes	Turn 7	5	$92.16	82.6	**Victory**
B	order → serve → seat	Yes	Yes	Never	5	$95.40	75.6	**Victory**
C	serve → order → seat	Yes	Yes	Turn 7	4	$67.86	84.3	**Victory**

D	seat → serve → order	No	Yes	Turn 7	4	$67.86	81.0	**Victory**
E	seat → serve → order	No	**No**	Turn 7	3	$49.50	68.7	Defeat
F ***(our script)***	seat → serve → order	No	No	**Turn 14**	3	$49.50	57.0	Defeat

Comparing rows that differ in one choice shows what each decision is worth.

Compare	The one change	Effect	Why
A vs C	Order first instead of serve first	5 paid instead of 4	With the Host seating, the Waiter can start orders sooner. In Row C, Elif is still waiting to pay when the game ends
D vs E	No Host	Victory becomes defeat	In Row E, Dev is still waiting to pay when the game ends, leaving only 3 paid
E vs F	Happy Hour on turn 14 instead of turn 7	Average falls from 68.7 to 57.0, below the target	Ben finishes at 28 instead of 58, and Chloe at 75 instead of 80

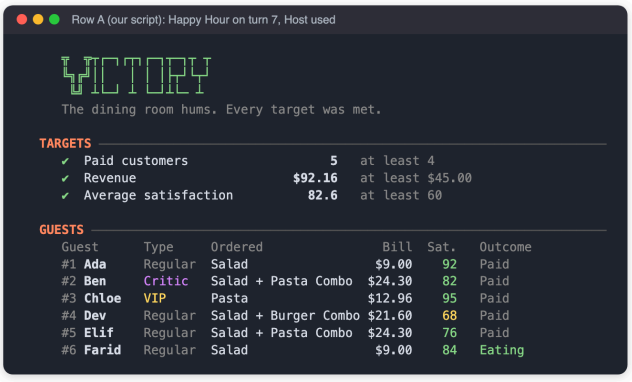


Figure 5.2: End screen for Row A (our victory script).
5 paid, $92.16 revenue, average satisfaction 82.6. All three targets are met.

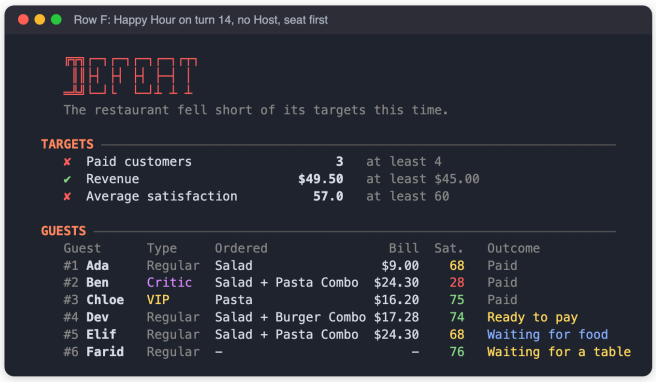


Figure 5.3: End screen for Row F (our defeat script).
3 paid and an average of 57.0 miss two of the targets. Ben finishes at 28, against 82 in Row A.

5.3 Zooming in: what Happy Hour on turn 7 does to Chloe and Ben

Rows A and B differ only in the Happy Hour choice, the same decision shown in Sequence Diagram 1 (message 9). **Both rows win, so here it is a trade-off rather than the difference between winning and losing.**

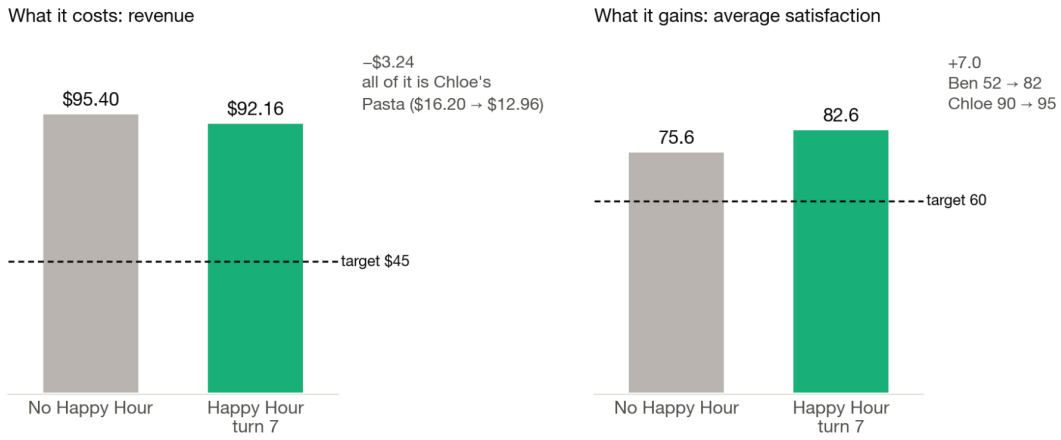


Figure 5.4: Happy Hour on turn 7 against no Happy Hour (Rows A and B).
It costs $3.24 in revenue, all of it from Chloe's Pasta, and raises the average satisfaction by 7.0.

Measure	Happy Hour on Turn 7 (Row A)	No Happy Hour (Row B)
Chloe's Pasta	$12.96	$16.20
Chloe's final satisfaction	95	90
Ben's final satisfaction	82	52
Total revenue	$92.16	$95.40
Average satisfaction	82.6	75.6

The cost: Chloe is the only customer who orders during turns 7–8, so the whole $3.24 difference is Chloe's discount.

The gain: Ben gains 30 (+15 on each turn). Chloe gains only 5 because satisfaction is capped at 100. That is 35 points across 5 paid customers, which raises the average by 7.0.

5.4 What we learned

Lesson	Evidence	Why, in the rules
Serve before you seat	With the same Chef timing, serve first averages 66.0 against 60.0 in Stage 1, and 64.0 against 62.5 in Stage 2. Seat → serve → order is the only priority that loses	A served customer stops losing satisfaction and frees the table two turns later. Seating first only moves the wait from the queue to a table
Don’t leave the Chef idle	Cooking on the same turn gives 5 paid instead of 4 in both stages	A dish cooked a turn later is served and paid for later, and customers behind it wait longer too
The Host changes the best priority	Row A beats Row C, and Row D wins where Row E loses	The Host seats with its own task, so the Waiter’s one task goes to ordering and serving

6 Development process and stage comparison

We built the game systematically, each member is assigned a particular section of the project and after completing their part, they would run ./test.sh and replayed that stage's victory and defeat scripts. This ensures each stage is functional before handing over to the next member to complete the next stage.

6.1 Commit History

Commit	Member	Tag	Content	Date
1	Apollo	-	Stage 1 (part 1): set up the project, main model (Money,menu,customers, orders, tables, restaurant), staff role and pricing.	23/10/2026
2	Leong Xuan Hao	stage-1	Stage 1 (part 2): Added turn controller, command-line interface, rule checks, victory and defeat runs.	26/10/2026
3	Lee Sheng Kai	stage-2	Stage 2: Added VIPCustomer and CriticCustomer	31/10/2026
4	Victoria Haugaa	stage-3	Stage 3: Added the Host, ComboMeal and the Happy Hour pricing policy.	01/11/2026

6.2 Stage 1 to Stage 2: Customer Behaviour

Class	Change	Why
VIPCustomer	new	A patient guest who pays less, loses 5 satisfaction (out of 100) per waiting turn instead of a regular's 8. The tradeoff for the delay of the VIP is billFor() gives 10% off.
CriticCustomer	new	An impatient guest loses 12 satisfaction per waiting turn and always orders the most complex dish (Pasta), so the least patient gust needs the most cooking, leaves a 1 to 5 star review upon payment.
Money	addition	percent() rounds a discount to the nearest cent.
GameConfig	registration	GameConfig is where the game is set up so new parts are registered there, VIPCustomer & CriticCustomer is added while retaining the normal workflow. Ben becomes CriticCustomer and, Chloe becomes a VIPCustomer.
Dashboard	Presentation	Shows each customer type in its own colour and explains the types in the help screen.
Game, Customer, Restaurant, Order and all staff	unchanged	The new customer behaviour is reflected through the customer objects, the remaining classes remain unchanged

Amendments made to earlier classes

- Money method
- Two registration lines in GameConfig (addition of VIPCustomer & CriticCustomer)
- A visual display change of unique colour code assigned to each customer type.

Stable rules

- No game rules rewritten in this stage

Design choice

- No interface added, since existing Customer class was enough

Extra feature

- The CriticCustomer's star review
- Dashboard display different colour for each customer type

Stage 2 to Stage 3: Customer Behaviour

Class	Change	Why
Host	new	Seats one queued customer per turn on top of the waiter's task.
Dish	new	The concrete dish with its name, price and units moved out of MenuItem
ComboMeal	new	Two dishes with their units summed and priced at 90% of their total rounded to nearest cent
PriceModifier, HappyHour	new	HappyHour can be switched on once per game for 2 turns, 20% off on orders taken and waiting gusts recover 15 satisfaction per turn. PriceModifier allows PricingPolicy apply it without knowing its rule.
MenuItem	rewritten	MenuItem was a single class for every dish, it is now the shared parent of Dish and ComboMeal, so Order,Chef and Waiter can handle a dish or combo the same way by asking for its price() and prepUnits()
PricingPolicy	rule change	Applies HappyHour after the customer's own discount (VIPcustomer 10% off first)
Game	Rule change	Runs the turn and enforce Happy Hour timing (Only once). Ensuring Happy hour can start only before any staff member acts, recovery at turn start.
Customer	addition	recover() adds the +15 Satisfaction during Happy Hour, capped at 100 , applicable to only guests still waiting
GameConfig, GameEvent	registration	GameConfig is where the game is set up so new parts are registered there, the new menu is built form three Dishes (MenuItem is abstract) and the two ComboMeal, and the Host is hired first so they act before the waiter. GameEvent labels each message shown to the player, Happy Hour's recovery message needs a new Policy label.
ConsoleUI, Dashboard	presentation	Asks whether to start Happy Hour and shows the turns left. Staff whose only move is wait now automatically waits.

DirectedStaff, Waiter,Chef, Cashier, Order, Restaurant	unchanged	Nothing changes since the Hosts reuses the existing code and comboMeal have the same methods as dishes, hence these classes work as before

Amendments made to earlier classes

- Money method
- Two registration lines in GameConfig (addition of VIPCustomer & CriticCustomer)
- A visual display change of unique colour code assigned to each customer type.
- MenuItem changed from a concrete class to an abstract one. Its public methods (name(),price,prepUnits()) stayed the same therefore Order,Chef and Waiter did not change. Only its constructor moved to Dish class, which is why GameConfig now builds Dish.

New rules

- No existing rule was rewritten, Happy Hour and the Host are new rules added.
- The host is always on duty and can seat one extra customer per turn, while the waiter keeps all of its stage 2 tasks.
- When Happy Hour is off, prices and satisfaction losses follow stage 2 rules exactly.
- HappyHour can be switched on once per game for 2 turns, 20% off on orders taken and waiting gusts recover 15 satisfaction per turn.

Design choice

- PriceModifier is the only interface added. It keeps Happy Hour's riles out of PricingPolicy and fixes the discount order in one place.
- The Host reuses the seating code in DirectedStaff.

Extra feature

- Auto-wait only applies when waiting is the staff member's only possible move, so the manager still makes every real choice and no required rule is replaced.

8 Rule Interpretation

Topic	Based on briefing guide	What our game does
Stage 1		
Food choice	Customers choose their own food by a documented, deterministic rule	Guest #n orders item n on the menu board. The choice is shown when the order is taken
Display order	Show the state at the start of the turn	The scheduled customer arrives before the dashboard is drawn
Who loses satisfaction	Waiting or seated customers who have not been served lose points at turn end	Waiting for a table, seated without ordering, and waiting for food (includes a READY dish not yet served).
Cashier, oldest comes first	The cashier handles one READY_TO_PAY customer per turn (oldest comes first)	Oldest means whoever became ready to pay earliest, a tie would go to the earlier arrival time.
Seating	The waiter seats a queued customer at a free table	The player can name the table or leave it out. Typing seat 2 seats customer #2 at the first free table, seat 2 1 seats customer #2 at table 1
Recorded satisfaction	The average uses the satisfaction recorded when each customer pays.	Before anyone has paid, the average shows N/A. After a customer pays, their satisfaction stays final and no longer changes.
Stage 2		
Which guests change type	Two regular arrivals may be replaced, there must still be 6.	Ben becomes CriticCustomer and Chloe becomes VIPCustomer.
The Critic's behaviour	A distinct order preference or reaction that you define.	Always orders the item with the most preparation units, the more expensive one on a tie and leaves a 1 to 5 star review when paying, which does not affect the score.
Stage 3		
Combos and combo choice	A combo is two existing items. Customers choose combos by a rule you document	The menu offers two fixed combos (Salad+Burger and Salad+Pasta) after the three dishes. Customers order a combo from the menu like any dish,

Host and waiter order	The host seats one customer in addition to the Waiter's task.	Host, then Waiter, then Chef. The Waiter may take the order of a guest the Host seated this turn.
"Start of a turn" for Happy Hour	Happy hour can be switched on at the start of a turn	Offered Happy Hour every turn until used, upon arrival and before any staff member acts. Recovery applies immediately and gain at the start of the second turn
Rounding	Round to the nearest cent, apply the VIP discount then subsequently the Happy Hour discount	Half -up after each percentage a VIP burger in Happy Hours goes $15.00,$13.50 and $10.80
Auto-wait	-	A staff member whose only possible action is to wait does so automatically without asking the player

Reflections (Lee Sheng Kai)

Contributions

I wrote the stage 2 code, which added VIPCustomer and CriticCustomer as subclasses of Customer. For the report, I made the UML class diagrams, section 6 (Development process and stage comparison) and section 8 (Rule interpretation)

What I learned

From the first half of the lectures I learnt what inheritance and polymorphism is, but i mostly known them for their definition. Stage 2 allowed me to apply the conceptual knowledge and see it work in real time, in which a well designed parent class allows you to add new things without touching the existing old codes. Initially when i was working on stage 2, i expected to make a lot of changes to the classes since new VIP customer gets a discount and Critics behave different from regular customers. In the end I only had to add 2 new classes for stage 2 since stage 1 already made Customer an abstract class with the necessary methods. Each new customer type just needs to override the parts that were different from regular customer and everything else remains the same. This was the exact moment where the concept of polymorphism clicked for me.

I also learned that composition and aggregation are not the same thing, even though both mean “has a”. Composition means the part only exists inside the whole and is created and thrown away with it. Aggregation means the part is only being used and can live on its own. I had never thought about this carefully until I had to draw it.

How I applied It

For VIPCustomer, it loses 5 instead of 8 satisfaction per waiting turn, billFor() gives 10% discount and the discount lives only within the VIPCustomer class. In order to keep money to the nearest whole cent, i added one method, percent() that rounds the bill to the nearest whole cent.
For CriticCustomer, we decided to have the Critic always order the dish that takes the most cooking and loses 12 satisfaction per waiting turn instead of 8. Since the addition of the two new type of customer does not change the original workflow in stage 1, other classes such as Game, Waiter etc all remained unchanged. Except for three minor changes that is Money,GameConfig and Dashboard as shown in section 6 Table 6.2.
In the class diagrams, I used the composition idea. Restaurant owns its Tables and Orders because they are created inside it and cannot exist without it. Customers are drawn with a

hollow diamond, because they are created by the arrival schedule and only pass through the restaurant.

What challenged me

The hardest part was not the code itself, but the documentation(class diagram, section 6 and 8). Especially the class diagram for the three stages, one such instance is when the MenuItem is a normal class in stage 1 and 2 but transitioned to be an abstract class in stage 3 which made it easy to mix them up.

How I solved it

To keep the diagrams right, I started with the class diagram for stage 1 and progressively made changes to it for subsequent stage 2 and 3. I checked the names and links of each diagram against their code for each stage and identified the change of normal class to abstract class of MenuItem in stage 3. For section 6(Development process and stage comparison) I referred to the codes and noted down the amendments made as the stage progress from 1 to 2 and 2 to 3, with explanation of what the changes are used for. For section 8(Rule interpretation) I referred to the assignment and briefing documents' criteria alongside playing the games at their respective stages to come up with the rule interpretation.

Reflections (Victoria Haugaa)

Contributions

My contributions included the implementation of stage 3 (i.e. The Host, Combo Meals and Happy Hour) and created the 6 sequence diagrams which follows VIP customer, Chloe, from her arrival on turn 7, along with relevant explanations to enhance readability.

What I learned and How I applied It

Before this project, I could simply define the OOP terms and create simple code based on the examples shown in the lecture notes. Doing this project and implementing stage 3 helped me further understand why these concepts mattered and how to apply them effectively.
I used to think of inheritance simply as a way to avoid repeatedly copying code that is common to certain classes. Making MenuItem abstract taught me that it is also about substitution. Dish and ComboMeal each implement the abstract methods price( ) and prepUnits( ), so waiter and PricingPolicy depend only on MenuItem, and dynamic dispatch selects the correct implementation at runtime. A Dish returns stored values while a ComboMeal is composed of two dishes and derives its values from them.
I also learned that an interface is also a “promise” about behaviour and not just a template for data. PriceModifier promises/declares adjust(price, turn) and HappyHour implements it, and PricingPolicy keeps a list of PriceModifier object and calls adjust on each modifier, ,so it never refers to HappyHour directly, i.e it depends on the interface rather than the concrete class. If I were to add a new discount I would write a new class that implements PriceModifier and add it to the policy’s list.
I also learnt the importance of encapsulation so that a class can enforce its own invariants. HappyHour.active throws a RuleViolation on a second call, and Order.serve rejects a dish that is not READY. Waiter and UI do not duplicate these checks and hence results in low coupling, which I have learnt means that classes depend little on each other’s internal implementation.
One design decision that stuck out most to me was Happy Hour. I did not put the discount inside Waiter.takeOrder, because that would tie pricing to rendering. Instead, I used a strategy pattern where each pricing rule is a separate object behind a common interface and can be swapped in without changing the code that uses it. PricingPolicy.quote first applies the customer’s own discount, then applies each PriceModifier such as Happy Hour. For example, a VIP Burger during Happy Hour costs $15 x 0.9 x 0.8 = $10.80. The price is fixed when the order is taken.

What challenged me

I found the sequence diagrams much harder to design than the actual implementation. The scenario spans turn 7 to turn 11 plus the result check after turn 18, which in total has over a 100 messages, and the relevant rules were distributed across several classes. In the diagrams, Ben hsa no lifeline but still affects the interactions such as which order the Waiter serves and who pays first. My initial singular diagram mixed these concepts and requirements, making the diagram incredibly large and unreadable.

How I solved it

To solve this, I decomposed the scenario into 6 parts, each tied to a specific requirement and added a walkthrough table mapping the messages to the behaviors to enhance readability. In the diagrams, I included only guard conditions that I was planning only implementing in the code such as game.canStartHappyHour(), and kept Ben in the notes only to limit the diagram's scope and reduce confusion. After the implementation of stage 3, I checked each message and value against the game's output, so that the diagram matches the final code and meets the project's requirements. Decomposing the scenario and having separate sequence diagrams helped to make the diagrams more meaningful and readable and it even made the code implementation easier.

Reflections (Poloyapoy Ron Joshua Abes (Apollo))

Contributions

I was the group leader and delegated tasks to the team and kept everything organised. I led the turn controller (`Game`, `GameConfig and ArrivalSchedule`) and the console UI and command parser. My key learning was keeping the UI thin. Because it never decides a rule, adding Stage 2 and Stage 3 barely touched it. I also edited the main demo video.

What I learned and How I applied it

Before this project I thought of separation of concerns as just putting files in different folders.

Building `Game` taught me what it actually means: `Game.beginTurn()` runs the six steps in order, but it never checks a price, a satisfaction value or customer type. It simply calls domain objects like `Restaurant.admit()` and `Cashier.collectPayment()` , and those objects enforce their own rules. When Stage 2 added VIP and Critic customers, `Game` did not change at all, because it was already calling `Customer` through the base class and the subclass behaviour took over at runtime. The same happened with `ConsoleUI` and `Dashboard` : they ask model objects for their display values rather than formatting rules themselves, so when the Dashboard needed to show customer types in colour for Stage 2, only `Dashboard` changed, and when Happy Hour needed a prompt, only `ConsoleUI` gained a few lines. I applied this by making sure that each new feature only touched the class that owned the data itself. For instance, `GameConfig` is the single place where the menu, customers and staff are registered, so adding the Host, combo meals and Happy Hour in Stage 3 meant adding registrations there and not rewriting the turn loop.

What challenged me

By far the hardest part was designing `GameConfig` and `ArrivalSchedule` so that each stage could change the game's setup without editing the controller. In Stage 1, there are four staff members and six Regular customers. In Stage 3, there are five staff (the Host is added), three customer types, five menu items instead of three, combo meals and Happy Hour. All of that must be declared somewhere, and if it is scattered across `Game`, the person adding Stage 3 would have to read and rewrite the turn loop. My first attempt method hard-coded the staff list inside `Game.beginTurn` which meant the method had to know every role.

How I solved it

I moved all per-stage setup into GameConfig: the turn count, the table count, the menu, the staff list and the arrival schedule. `Game` reads `GameConfig` once and then loops over whatever staff and events it finds. `ArrivalSchedule` maps turn numbers to customer factories, so a new customer type is just a new lambda in the config. Each stage extends the config by adding registrations (for instance, `GameConfig` in Stage 3 registers the Host before the Waiter so the Host acts first, and registers two `ComboMeal` entries on the menu). The result is that `Game` itself was unchanged from Stage 1 to Stage 3, which confirmed the design worked: new behaviour came from new objects, not from editing the controller.

Reflections (Leong Xuan Hao)

Contributions

I led the domain model (Order, Restaurant, Table and Money), the kitchen-to-payment flow, and the VIP and Critic customers. I also wrote rule checks for the order status rules and for the Critic's star rating at the boundary values. I learnt that when each rule sits in the object that owns the data, like Order.serve, you get clear error messages.

What I learned and How I applied It

Through this project, I learnt about how to use inheritance and polymorphism in my programming in order to make my code more structured and prevent subclasses from breaking important rules. Before, I thought that inheritance was mainly a way to avoid copying the same code repeatedly. However, through this project, I learnt that inheritance can help to enforce rules that every subclass must follow. For example, all Customers must follow the dining process, regardless of whether they are VIP, Critics or Regular customers. Although all customers must follow the same dining process, different customers can behave differently. For example, VIP might receive a discount, Critic loses more satisfaction and Regular customers pay the standard price. By keeping the main customer process inside the parent class and allowing the subclasses to customise specific behaviours, I was able to prevent different Customer types from accidentally breaking the program's logic.

What challenged me

The part that challenged me the most was giving the Critic special behaviour without making the main program dependent on the Customer's type. The Critic loses satisfaction faster, always orders the most elaborate dish (dish with the most preparation units, with ties going to the more expensive item), and leaves a star review when paying. At the start, I had wanted to use conditional statements to check the Customer's type inside the main game logic. However, I quickly realised that this would make the code increasingly complicated as more Customer types were added. It would also require modifying existing classes such as `Game` and `Waiter` whenever a new Customer type was added, going against the Open/Closed Principle.

How I solved it

Instead of placing all the Critic-specific behaviour inside of `Game` or `Waiter`, I had placed it inside of the `CriticCustomer` class. I implemented three specific behaviours for the Critic. `chooseFrom(Menu)` which selects the most elaborate dish, `satisfactionLossPerTurn()`, which is the amount of satisfaction the Critic loses per turn and `remarkOnPaying()` which is the star rating when the Critic is paying for their meal. These methods override the corresponding methods in the parent Customer class. `Waiter`, `Cashier` and `Game` don't need to check whether the customer is a Critic, VIP or Regular Customer, they just simply call the methods defined in the parent class and the program recognises that the actual object is `CriticCustomer` and the Critic's behaviour takes over at runtime. When adding Critic and VIP customer types, I only needed to create their subclasses and implement their special behaviours, I didn't need to rewrite existing classes such as: `Game`, `Restaurant` and `Order`. The only place that looks at the customer's type is a small switch in `Dashboard`, which is used for colour, and no game rule depends on it.